

M5-TTS: A Mini-Scale Multilingual and Multi-Dialect Text-to-Speech Language Model with Multi-Token Prediction

Wenhao Li[1,*], Jingbin Hu[1,*], Shengming Ling[1,*], Xiaming Ren[1,*], Haoyu Zhang[1,*], Jinrui Liang[1], Hanke Xie[1], Qirui Zhan[1], Guobin Ma[1], Huakang Chen[1], Chunbo Hao[1], Dake Guo[1], Yuepeng Jiang[1], Linhan Ma[1], Yunxiang Chen[2], Houdun Liu[2], Su Feng[2], Chuang Ding[3], Pengcheng Zhu[4], Liumeng Xue[5,†], Lei Xie[1,†]

[1]Audio, Speech and Language Processing Group (ASLP@NPU), School of Computer Science, Northwestern Polytechnical University
[2]Shenzhen Pimei Technology Co., Ltd., [3]Moonstep AI, [4]WeNet Open Source Community
[5]Audio, Speech and Language Processing Group (ASLP@NJU), School of Intelligence Science and Technology, Nanjing University

https://github.com/ASLP-lab/M5-TTS
https://huggingface.co/ASLP-lab/M5-TTS
https://aslp-lab.github.io/M5-TTS-Demo-Page
https://huggingface.co/ASLP-lab/M5-TTS-Space

Abstract

We present **M5-TTS**, a compact language-model-based text-to-speech system that unifies multilingual and Chinese multidialect synthesis, zero-shot voice cloning, speaker-specific multilingual generation, and efficient CPU streaming within a **50M-parameter** language model. M5-TTS employs a 16-codebook speech representation with multi-token prediction for efficient speech generation. To improve synthesis quality, we introduce a Group Relative Policy Optimization (GRPO) framework that integrates pause, word/character error rate (WER/CER), and speaker-similarity rewards. We further propose **CER-guided token-level credit assignment**, which concentrates optimization on localized pronunciation errors rather than uniformly penalizing entire sequences. On Seed-ZH and Seed-EN, M5-TTS achieves a CER of 1.37% and a WER of 2.91%, respectively, outperforming MOSS-TTS-Nano with only half the language-model parameters. On four cores of an Intel Xeon Gold 6530, M5-TTS achieves a real-time factor of 0.653 and a first-chunk latency of 225 ms, demonstrating practical CPU-only streaming synthesis. Subjective listening experiments suggest improved naturalness and expressiveness over Kokoro-TTS and so on. We will release the complete training, post-training, and inference pipelines, together with checkpoints for zero-shot voice cloning and a specific speaker, to facilitate reproducible research and the development of on-device speech generation and embodied voice interaction.

1 Introduction

Speech generation is a key component of spoken interaction for embodied agents, personal assistants, and intelligent devices. In these settings, a speech synthesizer needs to produce intelligible and natural speech, maintain a consistent voice, and begin playback with a short delay. Multilingual and dialectal capabilities further allow a single device to communicate with users across linguistic communities. Local inference can reduce dependence on network connectivity and give developers greater control over the speech generation pipeline. These requirements motivate compact text-to-speech (TTS) systems that

*These authors contributed equally to this work.
†Corresponding authors.

balance synthesis quality, linguistic coverage, voice flexibility, and streaming efficiency under limited computational resources.

Neural TTS research has explored diverse generation architectures. Early end-to-end approaches, such as VITS Kim et al. (2021), combine variational inference and adversarial learning for waveform generation, while non-autoregressive models such as FastSpeech 2 Ren et al. (2021)improve synthesis efficiency through parallel acoustic modeling and explicit duration, pitch, and energy conditioning. Nevertheless, these models can suffer from over-smoothing, which reduces fine-grained acoustic and prosodic variation and can limit the naturalness and expressiveness of synthesized speech.

Language-model-based TTS approaches use speech tokens as intermediate representations Wang et al. (2023), enabling autoregressive speech generation through sequence modeling. Subsequent studies have explored diverse speech representations and generation paradigms. CosyVoice 2 Du et al. (2024) combines autoregressive language modeling with conditional flow matching, while Qwen3-TTS Team (2026) employs multi-codebook tokens for multilingual and streaming synthesis. Recent work has also explored continuous speech generation without relying on conventional discrete tokens. DiTAR Jia et al. (2025) combines an autoregressive language model that captures dependencies across speech patches with a local diffusion Transformer that generates continuous acoustic features within each patch. VoxCPM2 Zhou et al. (2026) further explores tokenizer-free diffusion autoregressive modeling for multilingual speech generation and controllable voice cloning. Despite these advances, the substantial model sizes and inference costs of these systems can make low-latency streaming challenging on resource-constrained edge devices, particularly under CPU-only inference.

The growing demand for on-device voice interaction and embodied intelligence therefore creates a pressing need for compact TTS models that preserve naturalness and expressiveness while supporting efficient, responsive streaming synthesis. Recent compact TTS systems have further advanced on-device speech generation. MOSS-TTS-Nano OpenMOSS Team (2026)explores lightweight language-model-based speech generation with discrete codec tokens, while Pocket TTS Rouard et al. (2026) demonstrates efficient CPU-based inference with compact continuous models. Other lightweight projects, including Audio8-TTS Cai et al. (2026), Kokoro-TTS hexgrad (2025), Hojo-TTS-Light HojoAI (2026), TinyTTS Lê (2026), and sanoTTS Thapa (2026), further investigate reducing model complexity and computational requirements for resource-constrained speech synthesis.

Nevertheless, to the best of our knowledge, prior work has not yet demonstrated a unified TTS system that jointly supports multilingual and multidialect synthesis, zero-shot voice cloning, and CPU-based streaming within a 50M-parameter language model.

To address this gap, we present **M5-TTS**, a **mini-scale multilingual** and **multi-dialect** TTS **language model** with **multi-token prediction(MTP)** built around **50M** parameters, excluding the speech codec. M5-TTS supports zero-shot voice cloning and speaker-specific synthesis. The system combines a 16-codebook speech representation with MTP for efficient codec-token generation, and supports streaming inference on CPUs. Language, accent, and speaker conditioning provide explicit controls for synthesis, bringing linguistic diversity and voice customization into a compact generation framework. To improve generation quality, we employ Group Relative Policy Optimization (GRPO) Shao et al. (2024) with three complementary reward signals: a pause reward, a WER/CER reward, and a speaker-similarity reward. We further introduce CER-guided token-level credit assignment to focus pronunciation-related optimization on localized errors.

Our experiments examine synthesis quality and streaming performance, with MOSS-TTS-Nano OpenMOSS Team (2026) as the main comparison among lightweight language-model-based TTS systems. Using half as many language-model parameters, M5-TTS achieves 1.37% CER on Seed-ZH and 2.91% WER on Seed-EN, compared with 3.62% and 4.50% for MOSS-TTS-Nano, respectively. On four cores of an Intel Xeon Gold 6530, M5-TTS achieves a real-time factor of 0.653 and a first-chunk latency of 225 ms. Preliminary subjective listening experiments suggest improved naturalness and expressiveness over Kokoro-TTS and so on. We will release all training, post-training, and inference code, together with zero-shot voice-cloning and speaker-specific checkpoints, to support reproducible research and further community development. With this work, we aim to provide a practical speech generation component for on-device applications and embodied voice interaction.

2 Architecture

2.1 Tokenizer

We adopt Qwen-TTS-Tokenizer-12Hz from Qwen3-TTS Team (2026) to convert speech into discrete representations at 12.5 frames per second. Each frame contains 16 codebook indices, comprising one semantic codebook and 15 acoustic codebooks. The encoder consists of a downsampling convolutional

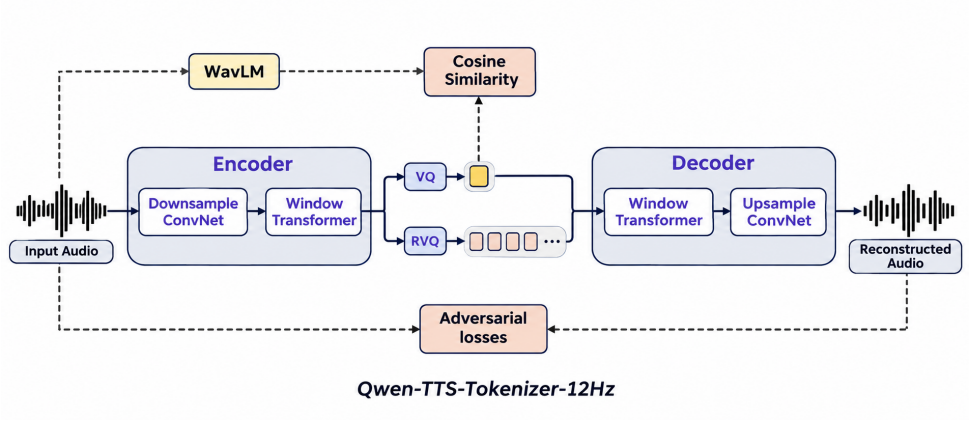


Figure 1: Qwen-TTS-Tokenizer-12Hz adopted by M5-TTS: a semantic VQ branch and an acoustic RVQ branch provide 16 codec tokens per frame for causal waveform reconstruction.

network followed by a windowed Transformer. Its latent features are quantized through a semantic VQ branch and a 15-stage acoustic RVQ branch, which represents complementary acoustic information. A windowed Transformer and an upsampling convolutional network then reconstruct the waveform from the quantized representations. In the tokenizer's original training framework, WavLM provides teacher features for semantic distillation of the first codebook using a cosine-similarity objective. Adversarial training encourages perceptually faithful reconstruction, while a multi-scale mel-spectrogram loss constrains spectral consistency. Both the encoder and decoder operate causally: speech tokens are extracted without look-ahead, and the corresponding waveform is reconstructed incrementally. This combination of a low frame rate and causal reconstruction provides M5-TTS with a compact speech representation suited to streaming generation.

2.2 Backbone

Following the hierarchical Transformer organization of Marvis-TTS Canuma & Newman (2025) and the multi-codebook prediction scheme of Qwen3-TTS Team (2026), M5-TTS uses a compact language-modeling architecture with 50M parameters, excluding the speech codec. The architecture comprises a causal backbone Transformer for temporal modeling and a lightweight local Transformer serving as the MTP module. Accent, speaker, and language IDs form an ordered conditioning prefix before the text sequence. For each historical audio frame, codebook-specific embeddings of its 16 codec tokens are summed into a single frame-level input, allowing the backbone to model speech at the codec frame rate.

Generation follows a hierarchical codebook prediction scheme. At each time step, the backbone summarizes the conditioning sequence and previously generated audio frames into a hidden state. A dedicated output head first predicts the zeroth codebook token. The backbone state and this token's embedding are projected to the MTP input space. The MTP module then autoregressively generates codebooks 1–15, with each prediction conditioned on the preceding codebooks within the same frame. The completed frame is aggregated and fed back to the backbone for the next temporal step.

During inference, the backbone retains its attention cache across audio frames, while the MTP cache is reset for each new frame. This factorization separates temporal modeling from within-frame acoustic refinement and avoids expanding all 16 codebooks into separate backbone time steps. Completed codec frames are passed to the causal waveform decoder for incremental reconstruction, supporting streaming synthesis.

3 Methods

3.1 Pre-training

3.1.1 Basic Pre-training

M5-TTS is pre-trained on approximately 400,000 hours of speech from multiple sources. The mixture includes Emilia and Emilia-YODAS He et al. (2024), covering Chinese, English, German, French, Japanese, and Korean, and approximately 160,000 hours of in-house data, and WenetSpeech4TTS, WenetSpeech-Yue, and WenetSpeech-Chuan Ma et al. (2024); Li et al. (2026); Dai et al. (2025). These sources provide multilingual and regional speech coverage within a shared model.

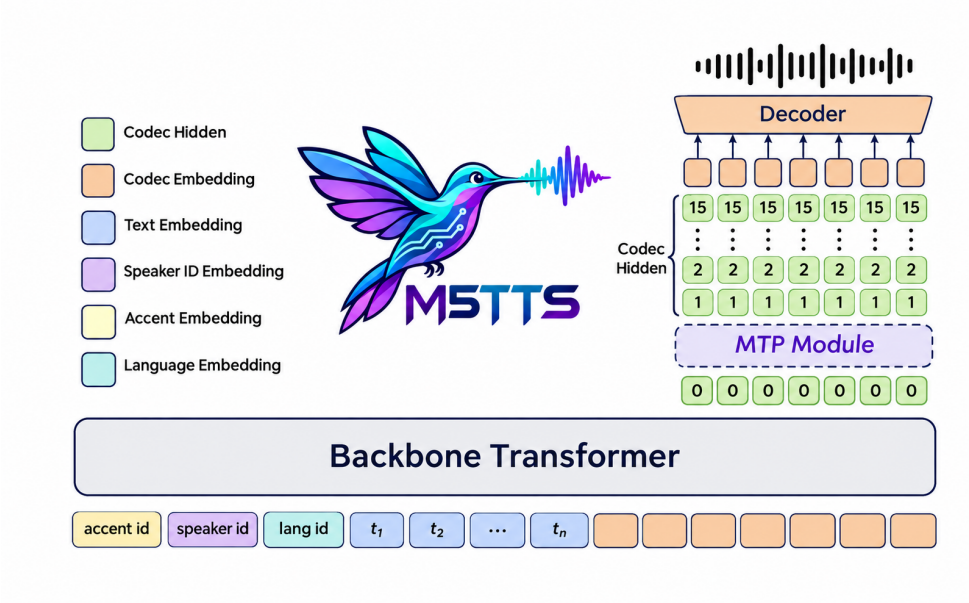


Figure 2: M5-TTS architecture. The temporal backbone predicts codebook 0; the local MTP module generates codebooks 1–15 within each frame, which are then decoded into speech.

The conditioning sequence follows the accent, language, and text order described in the Architecture section, and speaker ID will be added in the Speaker-Specific Fine-Tuning stage. Speech is represented by 16 codebook tokens per codec frame. Supervised training learns text–speech correspondences under the hierarchical autoregressive factorization: the temporal backbone predicts the zeroth codebook, and the local MTP module predicts the remaining codebooks conditioned on the backbone state and preceding within-frame tokens. Teacher forcing supplies the observed speech history during likelihood training. This establishes a general speech-generation policy for subsequent refinement of pronunciation, speaker consistency, and speaker-specific synthesis.

3.1.2 Continued Pre-training

To improve acoustic quality, expressiveness, and pronunciation robustness in multilingual and multidialectal synthesis, we perform continued pre-training (CPT) on a curated subset of the pre-training corpus. This subset comprises approximately 65,000 hours of speech, with selection prioritizing high-quality recordings, expressive speech, and challenging samples involving complex pronunciation patterns. The corpus covers Chinese, English, German, French, Japanese, and Korean, together with dialect data including Cantonese and Sichuanese. Language and dialect tags are encoded alongside the target text to provide explicit conditioning for speech generation. This stage aims to enhance speech quality and expressive delivery while strengthening the model's ability to handle complex pronunciations.

3.2 Post-training

3.2.1 Group Relative Policy Optimization

To encourage natural prosodic phrasing and prevent inappropriate pauses, we introduce a pause reward based on character-level temporal alignment. Specifically, we use the Qwen3-ForcedAligner Mu et al. (2026) to obtain timestamps for the generated speech and estimate the pause duration between adjacent characters. Given the aligned ending time $t_{i,j}^{\text{end}}$ of character j and starting time $t_{i,j+1}^{\text{start}}$ of the following character, we define the inter-character pause duration as

$$d_{i,j} = \max\left(0,\, t_{i,j+1}^{\text{start}} - t_{i,j}^{\text{end}}\right).$$

We distinguish between intra-sentence and punctuation-associated boundaries, as they exhibit different pause characteristics. For intra-sentence boundaries, excessively long pauses are penalized to encourage fluent speech. For punctuation-associated boundaries, we encourage pause durations within a predefined range to preserve appropriate prosodic segmentation.

Accordingly, the boundary-level pause reward is defined as

$$r_{i,j}^{\text{pause}} = \begin{cases} \mathbb{I}(d_{i,j} \le \tau_{\text{intra}}), & j \in \mathcal{B}_{\text{intra}}, \\ \mathbb{I}(\tau_{\text{punct}}^{-} \le d_{i,j} \le \tau_{\text{punct}}^{+}), & j \in \mathcal{B}_{\text{punct}}, \end{cases}$$

where $\mathcal{B}_{\text{intra}}$ and $\mathcal{B}_{\text{punct}}$ denote the sets of intra-sentence and punctuation-associated boundaries, respectively. The thresholds τ_{intra}, τ_{punct}^{-}, and τ_{punct}^{+} control the acceptable pause durations for the corresponding boundary types.

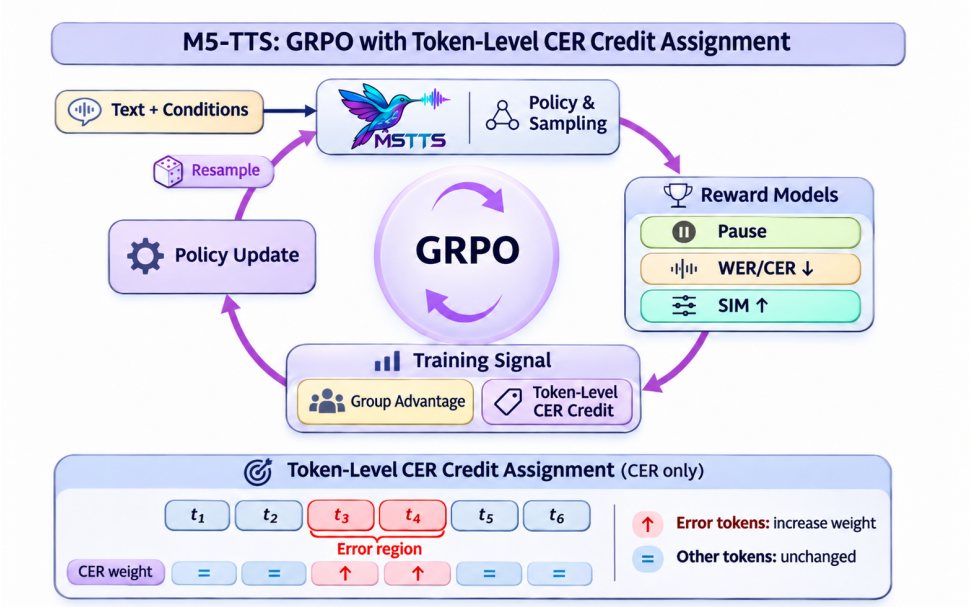


Figure 3: GRPO post-training with pause, WER/CER, and speaker-similarity rewards. CER-guided local credit assignment increases the weight of negative CER penalties in misread regions, while other token weights remain one.

The utterance-level pause reward is obtained by averaging over valid boundaries:

$$R_i^{\text{pause}} = \frac{1}{|\mathcal{B}_i|} \sum_{j \in \mathcal{B}_i} r_{i,j}^{\text{pause}},$$

where $\mathcal{B}_i$ contains the valid aligned boundaries for candidate i. This reward encourages fluent transitions within sentences while preserving appropriate pauses at punctuation boundaries.

To provide fine-grained feedback on content errors, we introduce a token-level CER credit assignment mechanism. For each generated speech candidate, we align its ASR transcription with the target text to identify character-level recognition errors, including substitutions, deletions, and insertions. The identified errors are then mapped to acoustic frames using forced alignment and character–frame correspondence.

We define a character-level error indicator $m_{i,j} = \mathbb{I}(e_{i,j} > 0)$, where $e_{i,j}$ denotes the number of edit operations assigned to target character j in candidate i. The error indicator is projected onto acoustic frames using overlap weights $w_{i,t,j}$, yielding $m_{i,t} = \sum_j w_{i,t,j} m_{i,j}$.

We then introduce an additional negative advantage for frames associated with recognition errors:

$$A_{i,t} = A_i^{\text{base}} - \lambda_{\text{local}} m_{i,t},$$

where A_i^{base} denotes the utterance-level group-relative advantage and λ_{local} controls the strength of the localized penalty.

This mechanism assigns stronger negative feedback to acoustic tokens associated with content errors while leaving the advantages of other tokens unchanged. Unlike utterance-level reward assignment, which distributes the same feedback across the entire generated sequence, our method explicitly penalizes error-associated generation decisions, enabling more localized optimization of speech content accuracy.

Thus, all generated frames receive the utterance-level feedback, while frames aligned with recognition errors receive an additional negative penalty. The frame-level advantage is shared by the acoustic codebooks associated with the same frame.

We jointly optimize content accuracy, speaker similarity, and pause naturalness using group-relative policy optimization. For each input, we sample G speech candidates from the rollout policy and evaluate them using three reward signals: character error rate (CER), speaker similarity (SIM), and pause reward.

We define the utterance-level rewards as

$$R_i^{\text{CER}} = -\text{CER}_i, \qquad R_i^{\text{SIM}} = \text{SIM}_i, \qquad R_i^{\text{pause}} = R_i^{\text{pause}}.$$

For each reward component $q \in \{\text{CER}, \text{SIM}, \text{pause}\}$, we compute its group-relative advantage:

$$A_i^q = \frac{R_i^q - \mu_q}{\sigma_q + \varepsilon},$$

where μ_q and σ_q are the mean and standard deviation of the corresponding rewards across the G candidates, and ε ensures numerical stability.

The utterance-level advantage is constructed as

$$A_i^{\text{base}} = \lambda_{\text{CER}} A_i^{\text{CER}} + \lambda_{\text{SIM}} A_i^{\text{SIM}} + \lambda_{\text{pause}} A_i^{\text{pause}},$$

where the coefficients control the relative contributions of the three reward components.

We optimize the generation policy using a clipped surrogate objective with KL regularization. For token $o_{i,t,c}$ from codebook c at frame t, we define the probability ratio as

$$r_{i,t,c}(\theta) = \frac{\pi_\theta(o_{i,t,c} \mid h_{i,t,c})}{\pi_{\text{old}}(o_{i,t,c} \mid h_{i,t,c})},$$

where π_{old} denotes the rollout policy used to sample the candidates and $h_{i,t,c}$ denotes the corresponding conditioning context.

The clipped token-level policy loss is

$$\mathcal{L}_{\text{clip},i,t,c}(\theta) = -\min\Big(r_{i,t,c}(\theta) A_{i,t}, \\ \text{clip}\left(r_{i,t,c}(\theta), 1-\epsilon_-, 1+\epsilon_+\right) A_{i,t}\Big),$$

where ϵ_- and ϵ_+ are the clipping margins. All codebooks within the same frame share $A_{i,t}$, while probability ratios and clipping are computed independently for each codebook.

The overall objective is

$$\boxed{\mathcal{L}_{\text{GRPO}}(\theta) = \mathcal{L}_{\text{clip}}(\theta) + \beta \mathcal{L}_{\text{KL}}(\theta),}$$

where $\mathcal{L}_{\text{clip}}$ aggregates the token-level clipped policy losses, and $\mathcal{L}_{\text{KL}}$ regularizes the policy against a frozen supervised fine-tuning (SFT) reference model. The coefficient β controls the regularization strength.

For our multi-codebook acoustic representation, the first-codebook loss is normalized by the number of generated decisions, while the depth-codebook losses are weighted across codebooks on sampled frames and normalized by the sampled-frame count. The two components are added to form the policy loss. The same aggregation scheme is applied to the token-level KL penalties. Advantages and reference-model predictions are detached during optimization.

3.2.2 Speaker-Specific Fine-Tuning

Specific-speaker fine-tuning adapts the shared model to curated recordings of a designated speaker. The text, language, accent, and speaker conditioning interface is retained, allowing supervised adaptation within the existing token-prediction framework. This supports a custom voice with consistent identity

and delivery. Curated speaker recordings, from the M2VoC Challenge 2021 Xie et al. (2021), provide speaker-specific supervision without requiring a separate speech representation.

This setting differs from voice cloning at inference time: it evaluates an adapted voice rather than the ability to match a supplied reference speaker. We therefore report custom-voice and voice-cloning results separately. Improvements for an adapted voice do not by themselves establish equivalent gains for unseen speakers.

3.3 Efficiency Optimization

To enable low-latency streaming speech synthesis on both CPUs and GPUs, we optimize the inference pipeline of M5-TTS without modifying its hierarchical autoregressive architecture. Our optimizations focus on four aspects: computation graph optimization, stateful inference and cache reuse, adaptive streaming scheduling, and hardware-aware runtime optimization.

3.3.1 Computation Graph Optimization

We reduce inference overhead by restructuring the execution graphs of the backbone and depth decoder. Specifically, we encapsulate the prediction and sampling of all 16 audio codebooks within a single depth-frame computation graph, reducing Python-level iteration and runtime invocation overhead. Importantly, the autoregressive dependencies among codebooks are preserved rather than replaced with parallel prediction.

We further fuse the query, key, and value projections in attention layers, as well as the gate and up projections in feed-forward layers, to reduce redundant operations and intermediate memory accesses. In addition, we precompute the linear projections of audio embeddings, replacing repeated embedding projection operations with efficient lookup-table accesses.

3.3.2 Stateful Inference and Cache Reuse

To eliminate redundant computation during autoregressive generation, we introduce explicit KV-cache management for both the backbone and depth decoder. On CPUs, these states are explicitly passed between ONNX Runtime inference calls, enabling incremental decoding and independent ONNX deployment.

We also implement a two-level reference caching mechanism. Reference audio is first encoded into reusable codec tokens, while the CPU inference pipeline additionally caches decoder states associated with complete reference inputs. A bounded LRU policy controls cache memory usage.

For streaming waveform generation, we implement a stateful codec decoder that maintains causal convolution histories, transposed-convolution overlap states, and sliding-window attention KV caches. This allows subsequent decoding steps to process newly generated codec tokens while reusing historical states, reducing redundant computation across overlapping audio chunks.

3.3.3 Adaptive Streaming Scheduling

We adopt a latency-aware streaming strategy that separates initial-chunk generation from subsequent decoding. A smaller initial chunk is used to reduce time to first audio, while larger decoding batches improve throughput during continuous generation.

For CPU inference, the decoding batch size is dynamically selected from 4, 8, and 16 frames according to the amount of generated audio available ahead of playback. This adaptive scheduling strategy balances first-chunk latency, overall real-time factor, and playback continuity.

3.3.4 Hardware-Aware Runtime Optimization

We apply hardware-specific execution optimizations to improve inference efficiency across CPU and GPU platforms.

For CPU inference, we optimize ONNX Runtime execution through physical-core affinity, intra-operation thread tuning, single-threaded inter-operation scheduling, sequential execution, and graph optimization. We additionally reduce resource overhead through idle thread-pool suspension, memory-mapped embeddings, and preallocated generation buffers. The execution graph is selected according to the CPU and threading configuration.

For GPU inference, the TTS model operates in BF16 precision, while the codec decoder remains in FP32 to preserve numerical stability. We compile the depth-frame computation graph and codec decoding

operations using **torch.compile**, with separate compiled codec graphs for fixed input-window shapes.

Finally, we reduce CPU–GPU synchronization and memory allocation overhead by accumulating EOS indicators within each decoding chunk and checking them at chunk boundaries, while preallocating codec-token, single-frame input, and positional buffers. The fast inference configuration additionally employs scaled dot-product attention (SDPA) for efficient attention computation.

4 Experiments

4.1 Benchmarks and Metrics

We organize the evaluation around three benchmark families: Seed-TTS Eval, TTS-Multilingual-Test-Set, and WenetSpeech Dialect Bench.

Content accuracy is measured by character error rate (CER) or word error rate (WER), using the edit distance between an ASR transcription of the generated speech and the target text. For either unit of comparison, the error rate is $100 * (S + D + I)/N$, where S, D, and I denote substitutions, deletions, and insertions, and N is the number of reference units. We use CER for Chinese, Japanese, and Korean and WER for English, German, and French. Speaker similarity (SIM) measures consistency with the reference voice, and higher values are preferred.

Systems and settings. We distinguish Voice Cloning and Custom Voice configurations throughout. The latter uses a specific speaker configuration and is not a matched zero-shot cloning comparison. Additional lightweight baselines are included where measurements are available. Missing entries indicate that a system does not support a capability. Bold model names identify M5-TTS.

4.2 Objective Results

Table 1: Structure of the evaluation benchmarks. CER and WER are reported in percent.

Benchmark	Subset	Language(s)	Content metric ↓	Speaker metric ↑
Seed	Seed-ZH	ZH	CER (%)	SIM
	Seed-EN	EN	WER (%)	SIM
	Seed-Hard	ZH	CER (%)	SIM
MiniMax	Multilingual	ZH, JA, KO	CER (%)	SIM
	Multilingual	EN, DE, FR	WER (%)	SIM
WenetSpeech	Chuan	Sichuanese	CER (%)	SIM
	Yue	Cantonese	CER (%)	SIM

Notes. ZH: Chinese; EN: English; DE: German; FR: French; JA: Japanese; KO: Korean.

Seed-TTS evaluation. Table 2 shows that M5-TTS (GRPO) achieves 1.37% CER on Seed-ZH and 2.91% WER on Seed-EN, compared with 3.62% and 4.50% for MOSS-TTS-Nano. These correspond to relative error reductions of 62.2% and 35.3%, respectively, with half the language-model parameters. SIM is also higher on these subsets, at 0.65/0.53 versus 0.64/0.50. MOSS-TTS-Nano retains an advantage on Seed-Hard, with lower CER (15.77% vs. 17.48%) and higher SIM (0.63 vs. 0.59). Compared with the 170M-parameter Audio8-TTS baseline, M5-TTS achieves lower Seed-Hard CER (17.48% vs. 19.63%), but has higher error rates on Seed-ZH and Seed-EN and lower SIM on all three subsets.

Multilingual voice cloning. On MiniMax (Table 3), M5-TTS (GRPO) achieves lower content error rates than MOSS-TTS-Nano in five of six languages. The relative reductions are 42.7%, 65.9%, 56.3%, 75.9%, and 64.9% for English, German, French, Japanese, and Korean, respectively. The largest reduction is observed in Japanese, where CER decreases from 40.22% to 9.70%. Chinese remains a gap, with 2.05% CER versus 1.80% for MOSS-TTS-Nano. SIM is higher in German, French, and Japanese, but lower in Chinese, English, and Korean. Pocket TTS achieves lower error rates in its reported English, German, and French voice-cloning evaluations, while M5-TTS has higher SIM in English and German and matches French SIM at the reported precision.

Table 2: Results on Seed-TTS Eval. CER and WER are reported in percent. Lower CER/WER and higher SIM indicate better performance.

Model	Main Model Parameters	Setting	Seed-ZH		Seed-EN		Seed-Hard	
			CER↓	SIM↑	WER↓	SIM↑	CER↓	SIM↑
M5-TTS (CPT)	50M	Voice Cloning	1.70	0.64	3.60	0.53	20.92	0.57
M5-TTS (GRPO)	50M	Voice Cloning	1.37	0.65	2.91	0.53	17.48	0.59
M5-TTS	50M	Custom Voice	1.26	—	1.88	—	18.19	—
Audio8-TTS	170M	Voice Cloning	**1.21**	**0.68**	1.96	**0.57**	19.63	**0.63**
MOSS-TTS-Nano	100M	Voice Cloning	3.62	0.64	4.50	0.50	15.77	**0.63**
Pocket TTS	100M	Voice Cloning	—	—	**1.86**	0.47	—	—
Hojo-TTS-Light	80M	Voice Cloning	2.15	0.40	3.63	0.17	21.10	0.36
Pocket TTS	100M	Custom Voice[a]	—	—	1.90	—	—	—
Kokoro-TTS	82M	Custom Voice[b]	1.40	—	2.04	—	**6.33**	—
Hojo-TTS-Light	40M	Custom Voice[c]	1.78	—	2.97	—	22.70	—
Sano TTS	2.27M	Custom Voice[d]	2.42	—	2.40	—	7.08	—
Tiny TTS	1.6M	Custom Voice	—	—	3.30	—	—	—

Notes. — denotes an unavailable result or an unspecified setting. Setting distinguishes Voice-Cloning and Custom-Voice configurations. For models with multiple preset voices, Custom-Voice results are obtained using one selected voice per language, as specified below. Unmarked Custom-Voice models use a single fixed voice.
[a] The English voice is recorded as `alba`; this spelling is preserved from the evaluation record.
[b] The Chinese and English voices are recorded as `zf_xiaobei` and `af_heart`, respectively.
[c] The Chinese and English voices are recorded as `hojo_zh_f_01` and `hojo_en_m_02`, respectively.
[d] The Chinese and English voices are recorded as `Xiaoya` and `Amy large`, respectively.

Table 3: Multilingual results on MiniMax. CER and WER are reported in percent. Lower CER/WER and higher SIM indicate better performance.

Model	Setting	ZH		EN		DE		FR		JA		KO	
		CER↓	SIM↑	WER↓	SIM↑	WER↓	SIM↑	WER↓	SIM↑	CER↓	SIM↑	CER↓	SIM↑
M5-TTS (CPT)	Voice Cloning	2.54	0.70	3.67	0.62	4.60	**0.63**	8.15	0.51	12.71	**0.72**	3.10	0.66
M5-TTS (GRPO)	Voice Cloning	2.05	0.71	3.06	0.63	3.81	**0.63**	8.14	**0.53**	9.70	**0.72**	2.88	0.66
M5-TTS	Custom Voice	1.16	—	2.24	—	**0.70**	—	5.33	—	10.27	—	**2.09**	—
MOSS-TTS-Nano	Voice Cloning	1.80	**0.73**	5.34	**0.65**	11.18	0.54	18.64	0.51	40.22	0.70	8.21	**0.67**
Pocket TTS	Voice Cloning	—	—	2.41	0.61	3.50	0.54	6.71	**0.53**	—	—	—	—
Pocket TTS	Custom Voice[a]	—	—	1.99	—	4.14	—	7.88	—	—	—	—	—
Sano TTS	Custom Voice[b]	1.41	—	**1.85**	—	3.18	—	8.91	—	—	—	—	—
Kokoro-TTS	Custom Voice[c]	**1.03**	—	2.58	—	—	—	**4.79**	—	**3.72**	—	—	—

Notes. — denotes an unavailable result or an unspecified setting. ZH, JA, and KO use CER; EN, DE, and FR use WER. All values are rounded to two decimal places. Setting distinguishes Voice-Cloning and Custom-Voice configurations. For models with multiple preset voices, Custom-Voice results are obtained using one selected voice per language, as specified below. Unmarked Custom-Voice models use a single fixed voice.
[a] The English, German, and French voices are recorded as `alba`, `juergen`, and `estelle`, respectively.
[b] The Chinese, English, German, and French voices are recorded as `Xiaoya`, `Amy large`, `Thorsten`, and `Siwis`, respectively.
[c] The Chinese, English, French, and Japanese voices are recorded as `zf_xiaobei`, `af_heart`, `ff_siwis`, and `jf_alpha`, respectively.

Table 4: Results on the Easy and Hard subsets for Cantonese (Yue) and Sichuanese (Chuan).

Dialect	TTS Model	ASR Model	Easy		Hard	
			CER↓	SIM↑	CER↓	SIM↑
Yue	M5-TTS (GRPO)	U2pp-Conformer-Yue	**8.55**	0.47	**8.70**	0.51
	CosyVoice2-Yue	U2pp-Conformer-Yue	10.33	**0.62**	9.49	**0.67**
	Llasa-1B-Yue	U2pp-Conformer-Yue	10.89	0.51	12.78	0.55
Chuan	M5-TTS (GRPO)	FireRedASR-AED	5.08	0.58	**7.78**	0.59
	CosyVoice2-WSC	FireRedASR-AED	**4.28**	**0.64**	8.78	**0.65**

Effect of GRPO post-training. Relative to CPT, GRPO reduces error rates on all three Seed subsets: CER/WER decreases from 1.70% to 1.37% on Seed-ZH, from 3.60% to 2.91% on Seed-EN, and from 20.92% to 17.48% on Seed-Hard. The corresponding relative reductions are 19.4%, 19.2%, and 16.4%. On MiniMax, error rates decrease in all six languages, with the largest relative reduction in Japanese (23.7%); French changes only marginally, from 8.15% to 8.14% WER. Across both benchmarks, SIM is maintained or improved at the displayed precision.

Speaker-specific synthesis. The custom-voice variant achieves 1.26% CER on Seed-ZH and 1.88% WER on Seed-EN. Compared with the GRPO voice-cloning variant, it also has lower content error rates in five of six MiniMax languages, including 0.70% WER in German. The GRPO voice-cloning variant performs better on Seed-Hard and MiniMax Japanese. Custom-voice results reflect dedicated speaker configurations and are reported separately from zero-shot voice cloning.

Dialect synthesis. Table 4 evaluates Cantonese and Sichuanese, using the same ASR evaluator for all systems within each dialect. For Cantonese, M5-TTS achieves the lowest CER among the compared systems on both subsets, at 8.55% on Easy and 8.70% on Hard. Relative to CosyVoice2-Yue, these correspond to reductions of 17.2% and 8.3%; relative to Llasa-1B-Yue, the reductions are 21.5% and 31.9%. Its SIM is lower than both baselines on both subsets, indicating a remaining gap in speaker preservation. For Sichuanese, M5-TTS reduces Hard CER from 8.78% to 7.78% relative to CosyVoice2-WSC, an 11.4% relative reduction, but has higher Easy CER (5.08% vs. 4.28%) and lower SIM on both subsets. These results demonstrate strengths in content accuracy on Cantonese and the Sichuanese Hard subset, alongside remaining limitations in speaker similarity.

4.3 Subjective Evaluation

We compare M5-TTS with MOSS-TTS-Nano using three mean opinion score (MOS) metrics: N-MOS for perceived audio quality, S-MOS for voice-cloning similarity, and I-MOS for speech intelligibility. N-MOS assesses acoustic clarity and fidelity, S-MOS measures perceived resemblance to the reference speaker, and I-MOS evaluates how easily listeners can understand the synthesized speech. The evaluation involves 10 listeners, who assign scores from 1 to 5, with higher scores indicating better performance. We report the mean ratings for each dimension.

Table 5: Subjective evaluation involving 10 listeners. All MOS scores range from 1 to 5; higher is better.

Model	N-MOS↑	S-MOS↑	I-MOS↑
MOSS-TTS-Nano	**3.00**	2.85	2.65
M5-TTS	2.90	**3.05**	**2.95**

Notes. N-MOS evaluates audio quality; S-MOS evaluates speaker similarity; I-MOS evaluates intelligibility. — denotes scores to be added.

Preliminary listening observations suggest that M5-TTS exhibits stable speech generation and good resemblance to the reference speaker. However, its perceived audio quality remains below that of MOSS-TTS-Nano, indicating room for improvement in acoustic fidelity. These qualitative observations will be complemented by the quantitative MOS results in Table 5.

4.4 Efficiency Evaluation

We evaluate the inference efficiency of M5-TTS using two key metrics: first-chunk latency and real-time factor (RTF). First-chunk latency measures the elapsed time from the initiation of speech generation to the availability of the first audio chunk, reflecting the system's responsiveness in streaming synthesis. RTF is defined as the ratio of total synthesis time to the duration of the generated speech, measuring the overall computational efficiency of speech generation. An RTF below 1 indicates that speech can be synthesized faster than its playback duration, a key requirement for real-time applications.

On four cores of an Intel Xeon Gold 6530, M5-TTS achieves an RTF of 0.653 and a first-chunk latency of 225 ms. The low first-chunk latency enables prompt audio output, while the sub-unitary RTF demonstrates faster-than-real-time synthesis, with approximately 1.53 seconds of speech generated per second of computation. Together, these results highlight the efficiency of M5-TTS for low-latency streaming speech synthesis on CPU hardware.

5 Limitations and Future Work

The current 16-codebook speech representation introduces considerable inference overhead in the local autoregressive decoder. Repeated codebook-generation steps place substantial demands on memory bandwidth, limiting CPU inference efficiency despite the compact LM. We plan to investigate inference with eight codebooks to reduce local decoding overhead, while evaluating the resulting trade-off between computational efficiency and acoustic fidelity.

Audio quality also remains a limitation. The current system supports only 24 kHz output, and highly expressive synthesis can exhibit voice breaks and hoarse or muted segments. These artifacts reduce perceived audio quality and indicate that expressive acoustic generation requires further refinement. Future work will extend the system to support 48 kHz output and improve the robustness of acoustic generation under expressive speaking conditions.

Intelligibility remains challenging on difficult inputs. M5-TTS (GRPO) achieves a CER of 17.48% on Seed-Hard, compared with 15.77% for MOSS-TTS-Nano, indicating that its pronunciation robustness under challenging conditions is still insufficient. Addressing this gap requires continued refinement of the data pipeline and training-data mixture. We will improve data quality control, expand coverage of challenging pronunciation patterns, and refine sampling proportions across general and difficult examples to strengthen robustness while preserving performance on standard multilingual inputs.

We also plan to introduce explicit control over emotion, paralinguistic expressions, and speaking style, including controllable emotional delivery and nonverbal vocalizations such as laughter and sighs. These extensions aim to enable more expressive and context-appropriate speech generation for on-device applications and embodied voice interaction.

6 Conclusion

We presented M5-TTS, a compact speech generation framework built around a 50M-parameter language model, excluding the speech codec. Its hierarchical architecture combines a 16-codebook speech representation with multi-token prediction, while explicit accent, speaker, and language conditioning supports multilingual and multidialect synthesis, zero-shot voice cloning, and speaker-specific multilingual generation. GRPO post-training integrates pause, WER/CER, and speaker-similarity feedback, with CER-guided token-level credit assignment providing localized pronunciation supervision.

Experiments demonstrate strong content accuracy within this compact parameter budget. Using half the LM parameters of MOSS-TTS-Nano, M5-TTS achieves relative error reductions of 62.2% on Seed-ZH and 35.3% on Seed-EN, and lower content error rates in five of the six MiniMax languages. Compared with CPT, GRPO reduces content error rates across the reported Seed and MiniMax subsets while maintaining or improving speaker similarity at the reported precision. Dialect evaluations further show lower Cantonese CER than both compared baselines and improved performance on the Sichuanese Hard subset, although challenging Chinese utterances and dialectal speaker preservation remain areas for improvement.

On four cores of an Intel Xeon Gold 6530, M5-TTS achieves an RTF of 0.653 and a first-chunk latency of 225 ms, demonstrating faster-than-real-time synthesis with responsive streaming output in the evaluated CPU configuration. These results support M5-TTS as a compact speech generation component for interactive applications and motivate further deployment and evaluation on resource-constrained edge devices for embodied voice interaction.

7 Ethics Statement

Voice cloning and speaker-specific synthesis can support accessibility, language learning, and personalized interfaces, but can also facilitate impersonation, fraud, harassment, or misleading recordings. Use of M5-TTS should require appropriate authorization from the represented speaker and respect applicable identity, publicity, and intellectual-property rights. Synthetic speech should be disclosed where listeners could otherwise reasonably mistake it for an authentic recording. The system should not be used to attribute statements to a person deceptively or to evade identity verification.

Training data and reference recordings raise separate licensing and privacy concerns. Data use and redistribution should follow the permissions and restrictions of each source, including those governing derived checkpoints. Reference audio, transcripts, and speaker representations should be handled as potentially identifying information, with collection, retention, and sharing limited to their stated purpose. Local inference can reduce transmission to external services, but does not by itself guarantee privacy.

Coverage and synthesis quality may differ across languages, dialects, accents, and speaker groups. The reported benchmark results should not be treated as proof of equal performance or suitability for consequential applications.

A Implementation and Reproducibility Details

A.1 Conditioning and Codebook Dependencies

The conditioning prefix orders accent, speaker, and language identifiers before the text. The following listing preserves the special-token spellings; its line breaks are for readability only and are removed when concatenating the prefix. Braced fields denote values to substitute, rather than literal text to synthesize.

```
<|accent_begin|>{accent id}<|accent_end|>
<|speaker_start|>{speaker id}<|speaker_end|>
<|lang_begin|>{lang id}<|lang_end|>{text}
```

This prefix specifies the ordering of the three controls; it does not specify the full serialization of reference speech for voice cloning. Identifier inventories, text normalization, and reference-prompt construction should accompany the released inference implementation. No additional codec boundary tokens are implied by this notation.

Let $q_{t,c}$ denote codebook c at audio frame t, with $c \in \{0, \ldots, 15\}$. The temporal backbone consumes the conditioning sequence and completed earlier frames. Within a historical frame, the 16 codebook embeddings are summed into one frame-level input. Its hidden state h_t first predicts $q_{t,0}$; the local MTP Transformer subsequently generates $q_{t,1}, \ldots, q_{t,15}$ sequentially:

$$p(\mathbf{q}_t \mid h_t) = p(q_{t,0} \mid h_t) \prod_{c=1}^{15} p(q_{t,c} \mid h_t, q_{t,0:c-1}). \tag{1}$$

Thus, the residual codebooks are not 15 independent parallel predictions. The backbone cache persists across frames, whereas the local cache is reset for each new frame. The local Transformer predicts codec indices; the separate speech-codec decoder reconstructs the waveform. The 50M parameter count refers to the LM component and excludes that codec.

A.2 Pre-training and Continued Pre-training Hyperparameters

Table 6 summarizes the configurations used for pre-training and continued pre-training (CPT). Both stages share the same backbone, local decoder, tokenizer, and 16-codebook speech representation, and use BF16 precision on CUDA. CPT uses lower learning rates, a shorter learning-rate schedule, and a larger maximum token budget.

Table 6: Hyperparameter configurations for pre-training and CPT. Field names follow the training configuration.

Hyperparameter	Pre-training	CPT
`backbone_flavor`	`llama-30M`	`llama-30M`
`decoder_flavor`	`llama-20M`	`llama-20M`
`tokenizer`	`llama3`	`llama3`
`audio_num_codebooks`	16	16
`learning_rate`	1×10^{-4}	1×10^{-5}
`min_lr`	1×10^{-5}	1×10^{-6}
`scheduler_steps`	1,000,000	150,000
`num_warmup_steps`	4,000	1,000
`save_every`	500,000	15,000
`max_tokens`	13,000	25,000
`max_batch_size`	999	999
`pad_multiple`	16	16
`decoder_fraction`	0.125	0.125
`lambda_depth`	1.0	1.0
`device`	`cuda`	`cuda`
`precision`	`bf16`	`bf16`

References

Runyuan Cai, Yu Lin, Yiming Wang, and Xiaodong Zeng. Training a compact, strong, and realistic text-to-speech model, 2026. URL https://arxiv.org/abs/2610.14776.

Prince Canuma and Lucas Newman. Marvis-tts: Efficient real-time voice cloning with streaming speech synthesis. GitHub repository, 2025. URL https://github.com/Neywa-Labs/marvis-tts.

Yuhang Dai, Ziyu Zhang, Shuai Wang, Longhao Li, Zhao Guo, Tianlun Zuo, Shuiyuan Wang, Hongfei Xue, Chengyou Wang, Qing Wang, Xin Xu, Hui Bu, Jie Li, Jian Kang, Binbin Zhang, and Lei Xie. Wenetspeech-chuan: A large-scale sichuanese corpus with rich annotation for dialectal speech processing. *CoRR*, abs/2509.18004, 2025. doi: 10.48550/ARXIV.2509.18004. URL https://doi.org/10.48550/arXiv.2509.18004.

Zhihao Du, Yuxuan Wang, Qian Chen, Xian Shi, Xiang Lv, Tianyu Zhao, Zhifu Gao, Yexin Yang, Changfeng Gao, Hui Wang, Fan Yu, Huadai Liu, Zhengyan Sheng, Yue Gu, Chong Deng, Wen Wang, Shiliang Zhang, Zhijie Yan, and Jingren Zhou. Cosyvoice 2: Scalable streaming speech synthesis with large language models. *CoRR*, abs/2412.10117, 2024. doi: 10.48550/ARXIV.2412.10117. URL https://doi.org/10.48550/arXiv.2412.10117.

Haorui He, Zengqiang Shang, Chaoren Wang, Xuyuan Li, Yicheng Gu, Hua Hua, Liwei Liu, Chen Yang, Jiaqi Li, Peiyang Shi, Yuancheng Wang, Kai Chen, Pengyuan Zhang, and Zhizheng Wu. Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation. In *IEEE Spoken Language Technology Workshop, SLT 2024, Macao, December 2-5, 2024*, pp. 885–890. IEEE, 2024. doi: 10.1109/SLT61566.2024.10832365. URL https://doi.org/10.1109/SLT61566.2024.10832365.

hexgrad. Kokoro. GitHub repository, 2025. URL https://github.com/hexgrad/kokoro.

HojoAI. Hojo-tts-light. GitHub repository, 2026. URL https://github.com/HojoAI/Hojo-TTS-Light.

Dongya Jia, Zhuo Chen, Jiawei Chen, Chenpeng Du, Jian Wu, Jian Cong, Xiaobin Zhuang, Chumin Li, Zhen Wei, Yuping Wang, and Yuxuan Wang. Ditar: Diffusion transformer autoregressive modeling for speech generation. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net, 2025. URL https://proceedings.mlr.press/v267/jia25a.html.

Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 5530–5540. PMLR, 2021. URL http://proceedings.mlr.press/v139/kim21f.html.

Trọng Hiếu Lê. Tinytts. GitHub repository, 2026. URL https://github.com/tronghieuit/tiny-tts.

Longhao Li, Zhao Guo, Hongjie Chen, Yuhang Dai, Ziyu Zhang, Hongfei Xue, Tianlun Zuo, Chengyou Wang, Shuiyuan Wang, Xin Xu, Hui Bu, Jie Li, Jian Kang, Binbin Zhang, Ruibin Yuan, Ziya Zhou, Wei Xue, and Lei Xie. Wenetspeech-yue: A large-scale cantonese speech corpus with multi-dimensional annotation. In Sven Koenig, Chad Jenkins, and Matthew E. Taylor (eds.), *Fortieth AAAI Conference on Artificial Intelligence, Thirty-Eighth Conference on Innovative Applications of Artificial Intelligence, Sixteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2026, Singapore, January 20-27, 2026*, pp. 31627–31635. AAAI Press, 2026. doi: 10.1609/AAAI.V40I37.40429. URL https://doi.org/10.1609/aaai.v40i37.40429.

Linhan Ma, Dake Guo, Kun Song, Yuepeng Jiang, Shuai Wang, Liumeng Xue, Weiming Xu, Huan Zhao, Binbin Zhang, and Lei Xie. Wenetspeech4tts: A 12, 800-hour mandarin TTS corpus for large speech generation model benchmark. In Itshak Lapidot and Sharon Gannot (eds.), *25th Annual Conference of the International Speech Communication Association, Interspeech 2024, Kos, Greece, September 1-5, 2024*. ISCA, 2024. doi: 10.21437/INTERSPEECH.2024-2343. URL https://doi.org/10.21437/Interspeech.2024-2343.

Bingshen Mu, Xian Shi, Xiong Wang, Hexin Liu, Jin Xu, and Lei Xie. Llm-forcedaligner: A non-autoregressive and accurate llm-based forced aligner for multilingual and long-form speech. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*, pp. 25627–25638. Association for Computational Linguistics, 2026. doi: 10.18653/V1/2026.ACL-LONG.1174. URL https://doi.org/10.18653/v1/2026.acl-long.1174.

OpenMOSS Team. Moss-tts-nano. https://github.com/OpenMOSS/MOSS-TTS-Nano, 2026.

Yi Ren, Chenxu Hu, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. Fastspeech 2: Fast and high-quality end-to-end text to speech. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=piLPYqxtWuA.

Simon Rouard, Manu Orsini, Axel Roebel, Neil Zeghidour, and Alexandre Défossez. Continuous audio language models. In Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust (eds.), *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/dd51dbce305433cd60910dc5b0147be4-Abstract-Conference.html.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

Qwen Team. Qwen3-tts technical report. *CoRR*, abs/2601.15621, 2026. doi: 10.48550/ARXIV.2601.15621. URL https://doi.org/10.48550/arXiv.2601.15621.

Ashish Thapa. sanotts: The smallest real-time neural tts on a general-purpose microcontroller, 2026. URL https://arxiv.org/abs/2608.21378.

Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, Lei He, Sheng Zhao, and Furu Wei. Neural codec language models are zero-shot text to speech synthesizers. *CoRR*, abs/2301.02111, 2023. doi: 10.48550/ARXIV.2301.02111. URL https://doi.org/10.48550/arXiv.2301.02111.

Qicong Xie, Xiaohai Tian, Guanghou Liu, Kun Song, Lei Xie, Zhiyong Wu, Hai Li, Song Shi, Haizhou Li, Fen Hong, Hui Bu, and Xin Xu. The multi-speaker multi-style voice cloning challenge 2021. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2021, Toronto, ON, Canada, June 6-11, 2021*, pp. 8613–8617. IEEE, 2021. doi: 10.1109/ICASSP39728.2021.9414001. URL https://doi.org/10.1109/ICASSP39728.2021.9414001.

Yixuan Zhou, Guoyang Zeng, Xin Liu, Xiang Li, Renjie Yu, Jiancheng Gui, Jiaheng Wu, Ziyang Wang, Xudong Shen, Runchuan Ye, Zhisheng Zhang, Jiuyang Zhou, Bingsong Bai, Weiyue Sun, Mengyuan Deng, Qundong Shi, Zhiyong Wu, and Zhiyuan Liu. Voxcpm2 technical report. *CoRR*, abs/2606.06928, 2026. doi: 10.48550/ARXIV.2606.06928. URL https://doi.org/10.48550/arXiv.2606.06928.